

P001 | Reconocer polimorfismo estático

Lee el siguiente código y responde las preguntas SIN ejecutarlo.

C++

```
class Impresora {
public:
    void imprimir(int n) {
        cout << "Entero: " << n << endl;
    }
    void imprimir(double d) {
        cout << "Double: " << d << endl;
    }
    void imprimir(string s) {
        cout << "Texto: " << s << endl;
    }
    void imprimir(int a, int b) {
        cout << "Suma: " << a + b << endl;
    }
};

int main() {
    Impresora imp;
    imp.imprimir(42);
    imp.imprimir(3.14);
    imp.imprimir(string("hola")); // Nota: usa string() explícito
                                   // "hola" solo es const char*
                                   // y puede ser ambiguo con bool
    imp.imprimir(3, 4);
    return 0;
}
```

Preguntas a resolver:

a) Escribe la salida exacta del programa.

Entero: 42

Double: 3.14

Texto: hola

Suma: 7

b) ¿Cuántas versiones de imprimir() existen? ¿Qué las diferencia?

Existen 4 versiones. Las diferencia la cantidad y el tipo de parámetros que reciben. Esto se conoce como sobrecarga de funciones

c) ¿En qué momento decide el compilador cuál versión llamar en cada línea de main?

Lo decide en tiempo de compilación (*early binding* o enlace estático). El compilador analiza los argumentos pasados en el main y los empareja con la función que mejor se ajuste a esos tipos

d) Agrega una quinta versión: imprimir(bool b) que imprima "Verdadero" o "Falso" según el valor. Escribe solo la firma y el cuerpo del método nuevo.

NOTA: Después de agregar imprimir(bool), ¿por qué imp.imprimir("hola") sin el cast explícito a string sería ambiguo?

El literal "hola" es de tipo const char* (un puntero). Al no existir una función exacta que reciba const char*, el compilador busca conversiones implícitas. Un puntero se puede convertir implícitamente a bool (si no es nulo es true) o a string. Al tener ambas opciones disponibles con el mismo nivel de prioridad, el compilador no sabe cuál elegir y genera un error de ambigüedad.

```
void imprimir(bool b) {  
    cout << (b ? "Verdadero" : "Falso") << endl;  
}
```

P002 | TRAZA DOBLE — con y sin virtual

RESUELVE A MANO. Este ejercicio muestra la diferencia central entre polimorfismo estático y dinámico.

C++

```
#include <iostream>  
using namespace std;
```

```
class Forma {  
public:  
    void dibujar() { // SIN virtual  
        cout << "Forma::dibujar" << endl;  
    }  
};
```

```

class Circulo : public Forma {
public:
    void dibujar() {
        cout << "Circulo::dibujar" << endl;
    }
};

```

```

void probar(Forma* f) {
    f->dibujar();
}

```

```

int main() {
    Forma fo;
    Circulo ci;
    fo.dibujar();    // llamada 1
    ci.dibujar();    // llamada 2
    probar(&fo);     // llamada 3
    probar(&ci);     // llamada 4
    return 0;
}

```

Preguntas a resolver:

a) Escribe la salida del programa TAL COMO ESTÁ (sin virtual).

Forma::dibujar

Circulo::dibujar

Forma::dibujar

Forma::dibujar

b) Ahora agrega virtual antes de void dibujar() en la clase Forma. ¿Qué cambia en la salida?

¿Qué llamadas se ven afectadas y cuáles no?

Forma::dibujar

Circulo::dibujar

Forma::dibujar

Circulo::dibujar

Cambia la llamada 4. Ahora detecta que el objeto real apuntado es un Circulo y ejecuta su método. Las llamadas 1, 2 y 3 no se ven afectadas.

c) Explica por qué la llamada 3 y la llamada 4 se comportan distinto según si hay virtual o no. La función probar recibe un puntero de la clase base (Forma* f).

SIN virtual: El compilador usa enlace estático. Se guía únicamente por el tipo del puntero (Forma*), por lo que ambas llamadas ejecutan Forma::dibujar.

CON virtual: Se activa el enlace dinámico. El programa en tiempo de ejecución revisa el puntero oculto (vptra) del objeto real para buscar su tabla virtual (vtable). En la llamada 4, el objeto real es un Circulo, por lo que redirige la llamada a Circulo::dibujar.

d) ¿Qué tipo de binding (enlace) usa cada versión: estático o dinámico?

Versión sin virtual: Binding estático (en tiempo de compilación).

Versión con virtual: Binding dinámico (en tiempo de ejecución) para las llamadas a través de punteros o referencias.

P005 | TRAZA — Slicing, el error silencioso

RESUELVE A MANO. No ejecutes hasta haber respondido.

C++

```
#include <iostream>
using namespace std;
```

```
class Figura {
public:
    virtual string nombre() const { return "Figura"; }
    void describir() const { cout << nombre() << endl; }
};
```

```
class Rectangulo : public Figura {
public:
    string nombre() const override { return "Rectangulo"; }
};
```

```
void porValor(Figura f) {          // Por VALOR
    f.describir();
}
```

```

void porPuntero(Figura* f) {    // Por PUNTERO
    f->describir();
}

void porReferencia(Figura& f) { // Por REFERENCIA
    f.describir();
}

int main() {
    Rectangulo r;
    porValor(r);    // línea 1
    porPuntero(&r); // línea 2
    porReferencia(r); // línea 3
    return 0;
}

```

Preguntas a resolver:

a) Escribe la salida de cada línea SIN ejecutar.

Línea 1 (porValor): Figura

Línea 2 (porPuntero): Rectangulo

Línea 3 (porReferencia): Rectangulo

b) ¿En cuál(es) líneas ocurre slicing? ¿Por qué?

Ocurre en la Línea 1. Porque la función porValor(Figura f) recibe el parámetro por valor. Al pasarle el objeto r de tipo Rectangulo, se crea una copia recortada que solo conserva la parte de Figura, perdiendo el enlace al método sobrescrito

c) ¿Cuál es la diferencia entre la línea 2 y la línea 3? ¿Cuándo preferirías usar referencia sobre puntero?

La línea 2 usa la dirección de memoria explícita (&r) y desreferenciación mediante flecha (->), mientras que la línea 3 pasa el objeto de forma directa y limpia usando alias internos. Es preferible usar la referencia cuando el objeto no necesita ser nulo (nullptr) y quieres un código más limpio, seguro y legible sin lidiar con la sintaxis de punteros.

d) Modifica la llamada de la línea 1 para que NO produzca slicing. Escribe la nueva firma de porValor().

Para que no produzca slicing conservando el paso de parámetros original del main, se debe cambiar la firma de la función para que reciba por referencia constante:

```
void porValor(const Figura& f) { // Al ser referencia, ya no hay slicing
    f.describir();
}
```

P008 | ANÁLISIS — Errores comunes con mensajes de compilador

Para cada fragmento: indica si compila, si el error es en compilación o en ejecución, y escribe la corrección correspondiente.

C++

// Fragmento A:

```
class Animal {
public:
    virtual void hablar() = 0;
};

int main() {
    Animal a;
    a.hablar();
}
```

// Fragmento B:

```
class Animal {
public:
    virtual void hablar() { cout << "..."; }
};

class Perro : public Animal {
public:
    void habla() override { cout << "Guau"; } // Typo (error de dedo)
};
```

// Fragmento C:

```
class Animal {
public:
```

```

    void hablar() { cout << "Animal"; }
    ~Animal() {} // NO virtual
};
class Perro : public Animal {
private:
    int* datos;
public:
    Perro() { datos = new int[100]; }
    ~Perro() { delete[] datos; }
};
int main() {
    Animal* a = new Perro();
    delete a; // ¿Qué pasa aquí?
}

```

// Fragmento D:

```

class Figura {
public:
    virtual double area() = 0;
};
class Circulo : public Figura {
    double radio;
public:
    Circulo(double r) : radio(r) {}
    // Olvidó implementar area()
};
int main() {
    Circulo c(5.0);
}

```

Respuestas a rellenar:

Fragmento A: No compila. Error en compilación.

Corrección A: No se puede instanciar la variable Animal a; directamente en el main porque Animal es una clase abstracta debido al método virtual puro = 0. Debe usarse mediante punteros, por ejemplo: Animal* a = new ...(con una clase hija que implemente hablar()).

Fragmento B: Compila, pero hay un error de lógica en compilación / diseño.

Corrección B: La clase Perro no está sobrescribiendo el método debido a un error de dedo (habla() en lugar de hablar()). El compilador se detendrá en override indicando que no pertenece a la base. Corregir el nombre a void hablar() override.

Fragmento C: Compila, pero produce un error crítico en tiempo de ejecución.

Corrección C: Al hacer delete a; siendo a un Animal*, solo se ejecuta el destructor de Animal, dejando los 100 enteros de Perro flotando en memoria (*memory leak*). Se corrige agregando virtual al destructor de la base: virtual ~Animal() {}.

Fragmento D: No compila. Error en compilación.

Corrección D: Al intentar instanciar Circulo c(5.0);, el compilador falla porque Circulo heredó un método virtual puro (area()) y no lo implementó, volviéndose abstracta también. Se corrige implementando la función dentro de Circulo: double area() override { ... }.

P011 | TRAZA — Cuando dynamic_cast retorna nullptr

RESUELVE A MANO primero. Dibuja la jerarquía y predice cada resultado antes de ejecutar

```
#include <iostream>
```

```
using namespace std;
```

```
class A {  
public:  
    virtual ~A() {}  
};  
class B : public A {};  
class C : public A {};  
class D : public B {};
```

```
int main() {  
    A* a1 = new B();  
    A* a2 = new C();  
    A* a3 = new D();  
  
    B* b1 = dynamic_cast<B*>(a1); // caso 1  
    B* b2 = dynamic_cast<B*>(a2); // caso 2  
    B* b3 = dynamic_cast<B*>(a3); // caso 3  
    D* d1 = dynamic_cast<D*>(a3); // caso 4
```

```
C* c1 = dynamic_cast<C*>(a3); // caso 5
```

```
cout << (b1 ? "OK" : "null") << endl; // línea 1
cout << (b2 ? "OK" : "null") << endl; // línea 2
cout << (b3 ? "OK" : "null") << endl; // línea 3
cout << (d1 ? "OK" : "null") << endl; // línea 4
cout << (c1 ? "OK" : "null") << endl; // línea 5
```

```
delete a1; a1 = nullptr;
delete a2; a2 = nullptr;
delete a3; a3 = nullptr;
return 0;
}
```

a) Árbol de herencia:

Plaintext

```
  [ A ] (Clase Base Polimórfica)
 /   \
[ B ] [ C ]
 |
[ D ]
```

b) Predicción de cada caso y justificación:

Caso 1 (b1): OK *Justificación:* a1 apunta a un objeto real de tipo B. El cast a B* es exacto y exitoso.

Caso 2 (b2): null *Justificación:* a2 apunta a un objeto de tipo C. Un C no hereda de B ni comparte esa rama, por lo que el cast de C hacia B* es imposible y falla.

Caso 3 (b3): OK *Justificación:* a3 apunta a un objeto real de tipo D. Dado que D deriva directamente de B, el objeto real contiene de forma válida la estructura de B.

Caso 4 (d1): OK *Justificación:* a3 apunta a un objeto real de tipo D. El cast a D* es exacto y exitoso.

Caso 5 (c1): null *Justificación:* a3 apunta a un objeto de tipo D. D se encuentra en la rama de B (A -> B -> D) y no tiene ninguna relación con la rama de C, por lo que el cast a C* falla.

c) *¿Por qué el caso 3 funciona aunque a3 se declaró como A?**

Porque dynamic_cast no analiza el tipo del puntero declarativo (A*), sino que utiliza el

mecanismo RTTI para inspeccionar el objeto real en la memoria heap. Al evaluar el objeto, descubre que es un D. Como D hereda de B, la conversión hacia arriba (upcasting indirecto desde el tipo real D hacia B) es totalmente válida dentro de su línea sucesoria.

